



Universidade Federal de Sergipe — Departamento de Computação
COMP0512 — Programação C · Turma T02 · 2026.2

Lista de Exercícios — Fundamentos, Ponteiros e Memória Dinâmica

Conteúdo trabalhado até aqui: fundamentos de C, tipos e TADs, `struct`, `enum/typedef`, ponteiros, vetores e strings, alocação dinâmica de memória, vetores e matrizes dinâmicos.

Prof. André Yoshiaki Kashiwabara · andreyoshiaki@dcomp.ufs.br

Parte I — Fundamentos, tipos e abstração

Exercício 1. Rastreio de laços e divisão inteira.

Considere o programa abaixo.

```
1 #include <stdio.h>
2
3 int main(void) {
4     int n = 5;
5     int soma = 0;
6
7     for (int i = 1; i <= n; i++) {
8         if (i % 2 == 0) continue;
9         soma += i;
10    }
11
12    int total = 0;
13    for (int i = 0; i < n; i++)
14        for (int j = i; j < n; j++)
15            total++;
16
17    printf("A: %d\n", soma);
18    printf("B: %d\n", total);
19    printf("C: %d\n", soma / 2);
20    printf("D: %.2f\n", soma / 2.0);
21    return 0;
22 }
```

- (a) Escreva as quatro linhas impressas.
- (b) Quantas vezes o corpo do laço interno (`total++`) executa? Dê a conta, não só o número.
- (c) As linhas **C** e **D** calculam a mesma divisão e imprimem valores diferentes. Explique por quê, citando o tipo de cada operando.

Exercício 2. Conversões e limites dos tipos.

Considere o trecho abaixo.

```
1 char c = 'A';
2 int i = c + 1;
3 double d = 7 / 2;
4 double e = 7 / 2.0;
5 int f = 3.9;
6 short s = 32767;
7 s = s + 1;
8
9 printf("%c %d %.1f %.1f %d %d\n", c, i, d, e, f, s);
```

- (a) Escreva a linha impressa.
- (b) Por que `c + 1` pode ser somado a um inteiro, se `c` é um caractere?
- (c) Explique, em uma frase, o valor de `f`: a conversão arredonda ou trunca?
- (d) O valor de `s` depende do compilador ou é garantido pela linguagem? Justifique e diga como você reescreveria a declaração de `s` para que o programa suportasse o valor 32768.

Exercício 3. Tipo, estrutura de dados e TAD: separar o *quê* do *como*.

Uma *fila de atendimento* de um laboratório guarda, em ordem de chegada, o nome e a matrícula dos alunos que pediram ajuda. Sobre ela, o resto do programa precisa poder: criar uma fila vazia, colocar um aluno no fim, retirar o primeiro, saber quantos estão esperando e saber se está vazia.

- (a) Escreva o arquivo de interface `fila.h` **apenas com** a definição dos tipos necessários e os *protótipos* das cinco operações. Não escreva nenhum corpo de função.
- (b) Descreva, em duas ou três frases, uma implementação possível (o *como*): que estrutura de dados você usaria por trás e quais campos ela teria.
- (c) Aponte um detalhe da sua implementação que o usuário do TAD **não** precisa conhecer, e explique o que se ganha ao escondê-lo.

Exercício 4. Funções sobre um vetor de notas.

Escreva, em C, as três funções abaixo. Nenhuma delas deve imprimir nada.

```
1 double media(const double v[], int n);
2 double maior(const double v[], int n);
3 int    conta_acima(const double v[], int n, double limite);
```

- (a) Implemente as três.
- (b) Escreva uma `main` que declare `double notas[] = {7.5, 4.0, 9.25, 6.0, 3.5};`, chame as três funções e imprima a média com duas casas decimais, o maior valor e quantas notas são maiores ou iguais a 6,0.
- (c) Por que o parâmetro é declarado `const double v[]` e não `double v[]`? O que o `const` promete a quem chama a função?
- (d) O que acontece se alguém chamar `media(notas, 0)`? Altere a sua implementação para que esse caso não produza uma operação inválida.

Parte II — Registros, enumerações e apelidos de tipo

Exercício 5. Mapa de bytes e reordenação de um registro.

Considere a declaração abaixo, adotando os tamanhos `char = 1`, `short = 2`, `int = 4`, `float = 4`, `double = 8` e ponteiro = 8 bytes, com alinhamento igual ao tamanho do tipo.

```
1 struct Registro {
2     char    sigla;
3     double  valor;
4     int     codigo;
5     char    nome[6];
6     short   ano;
7 };
```

- (a) Some, campo a campo, o tamanho declarado. Quantos bytes dá?

- (b) Desenhe o mapa de bytes do registro: uma faixa de 0 até `sizeof(struct Registro)-1`, marcando onde começa cada campo e onde há *padding*.
- (c) Diga o valor de `sizeof(struct Registro)` e de `offsetof(struct Registro, ano)`.
- (d) Quantos bytes de *padding* existem no total, e quais são as duas regras do compilador que os produziram?

Ainda sobre a mesma struct Registro:

- (e) Reescreva a declaração reordenando os campos de modo a obter o menor `sizeof` possível, sem remover nenhum campo.
- (f) Diga o novo `sizeof` e quantos bytes você economizou.
- (g) Existe alguma ordenação que chegue a exatamente 21 bytes? Justifique.
- (h) Cite uma situação, das discutidas em aula, em que essa economia importa de verdade — e uma em que não vale a pena reordenar.

Exercício 6. Registros em funções e vetor de registros.

Considere o tipo abaixo.

```
1 typedef struct {
2     char    nome[40];
3     int     matricula;
4     double  nota;
5 } Aluno;
```

- (a) Escreva `void aplicar_bonus(Aluno a, double b)` que soma `b` à `nota`, e `void aplicar_bonus_ref(Aluno *a, double b)` que faz o mesmo por ponteiro. Mostre as duas chamadas a partir de uma variável `Aluno x`; e diga qual das duas realmente altera `x`. Explique por quê.
- (b) Um registro é copiado inteiro quando passado por valor, mas um vetor não. Explique essa diferença em duas frases.
- (c) Escreva `int melhor(const Aluno turma[], int n)` que devolve o *índice* do aluno de maior nota (em caso de empate, o de menor índice), e `-1` se `n <= 0`.
- (d) Por que a função devolve o índice em vez de devolver o próprio `Aluno`? Cite uma vantagem de cada escolha.

Exercício 7. Enumerações: valores, switch e o truque do contador.

Considere a declaração abaixo.

```
1 typedef enum { NORTE, SUL, LESTE = 10, OESTE, NUM_DIRECOES } Direcao;
```

- (a) Escreva o valor inteiro de cada uma das cinco constantes.
- (b) Um colega escreveu `int contagem[NUM_DIRECOES] = {0};` e pretende usar `contagem[d]++` para contar ocorrências de cada direção. O vetor tem tamanho suficiente? Há desperdício? Explique o que acontece com esse código.
- (c) Reescreva a enumeração de forma que o truque do contador funcione sem desperdício, mantendo os quatro nomes de direção.
- (d) Escreva uma função `const char *nome_direcao(Direcao d)` usando `switch`, com um caso para cada direção. O que o compilador consegue avisar nessa função que ele não avisaria se as direções fossem `#define`?

- (e) Direcao d = 42; compila? O que isso mostra sobre a garantia que um `enum` oferece?

Exercício 8. typedef: ler declarações e reconhecer armadilhas.

Responda:

- Escreva as **três** formas de declarar um registro `Ponto` com campos `double x`, `y` usando `struct` e `typedef` (com nome de *tag*, sem *tag*, e as duas juntas). Diga qual delas permite escrever `struct Ponto p`; e qual permite `Ponto p`.
- Dado `typedef double Vetor3[3];`, diga o que é declarado por `Vetor3 a`; e o que o parâmetro `void f(Vetor3 v)` realmente recebe. Por que essa forma é considerada uma armadilha?
- `typedef` cria um tipo novo ou um apelido? Mostre um exemplo curto em que essa diferença aparece (isto é, em que o compilador *não* reclama de algo que você gostaria que ele reclamasse).

Parte III — Ponteiros

Exercício 9. Rastreio com &, * e aritmética de ponteiros.

Considere o programa abaixo.

```

1  #include <stdio.h>
2
3  void dobra(int *p, int n) {
4      for (int i = 0; i < n; i++)
5          *(p + i) = *(p + i) * 2;
6  }
7
8  int main(void) {
9      int v[5] = {1, 2, 3, 4, 5};
10     int *p = v;
11     int *q = &v[3];
12
13     printf("A: %d %d\n", *p, *(p + 2));
14     printf("B: %ld\n", (long)(q - p));
15     p = p + 1;
16     printf("C: %d\n", *p);
17     dobra(v, 5);
18     printf("D: %d %d %d\n", v[0], *p, *q);
19     return 0;
20 }
```

- Escreva as quatro linhas impressas.
- Suponha que `v` comece no endereço `0x100`. Qual é o valor numérico de `p + 2` na linha **A**? Por que não é `0x102`?
- A linha **B** imprime uma diferença de ponteiros. O que esse número significa: bytes ou elementos?
- Se a chamada fosse `dobra(p, 4)` em vez de `dobra(v, 5)`, o que a linha **D** imprimiria? Explique.

Exercício 10. O bug do troca que não troca.

Um colega escreveu as duas versões abaixo e não entende por que só uma funciona.

```

1  void troca_v1(int a, int b) {
2      int t = a;
3      a = b;
```

```

4     b = t;
5 }
6
7 int main(void) {
8     int x = 10, y = 20;
9     troca_v1(x, y);
10    printf("%d %d\n", x, y);
11    return 0;
12 }

```

- O que o programa imprime? Explique, falando de *cópias* de argumentos.
- Desenhe a memória durante a execução de `troca_v1`: as variáveis de `main` de um lado, os parâmetros da função do outro.
- Escreva `troca_v2` usando ponteiros e mostre a chamada correta a partir de `main`. Onde entra o `&` e onde entra o `*`?
- Seu colega tentou `troca_v2(&x, &x)`. O que acontece? A função quebra?

Exercício 11. Percorrer sem colchetes.

Escreva as funções abaixo **sem usar o operador []** em nenhum lugar — apenas ponteiros e aritmética de ponteiros.

```

1 int soma_v(const int *v, int n);
2 void inverte(int *v, int n);
3 int *primeiro_negativo(int *v, int n);

```

- Implemente as três. A terceira devolve o *endereço* do primeiro elemento negativo, ou `NULL` se não houver nenhum.
- Em `inverte`, use dois ponteiros que caminham em sentidos opostos e param quando se cruzam. Escreva a condição de parada do laço.
- Explique por que `v[i]` e `*(v + i)` são a mesma coisa para o compilador.
- Se `primeiro_negativo` devolvesse o índice em vez do endereço, como o chamador distinguiria “não encontrei”? Compare as duas soluções.

Exercício 12. Ponteiro para ponteiro.

Considere o trecho abaixo.

```

1 int x = 10, y = 20;
2 int *p = &x;
3 int **pp = &p;
4
5 **pp = 30;
6 printf("A: %d %d\n", x, *p);
7 *pp = &y;
8 **pp = 40;
9 printf("B: %d %d %d\n", x, y, *p);

```

- Escreva as duas linhas impressas.
- Desenhe a cadeia de endereços: `pp` → `p` → variável. Mostre o desenho *antes* e *depois* da linha `*pp = &y;`.
- Qual é a diferença entre alterar `*pp` e alterar `**pp`?
- Cite uma situação prática, vista em aula, em que um parâmetro `int **` é necessário.

Parte IV — Vetores, strings e passagem por referência

Exercício 13. O vetor que encolhe ao entrar na função.

Considere o programa abaixo.

```

1 #include <stdio.h>
2
3 void mostra(int v[]) {
4     printf("dentro: %zu\n", sizeof v);
5 }
6
7 int main(void) {
8     int v[10] = {0};
9     printf("fora: %zu\n", sizeof v);
10    mostra(v);
11    return 0;
12 }
```

- Escreva as duas linhas impressas e explique a diferença.
- Escreva as **três** declarações equivalentes do parâmetro de `mostra` e diga por que as três significam exatamente a mesma coisa para o compilador.
- Reescreva `mostra` de modo que ela consiga imprimir quantos elementos o vetor tem. O que precisou mudar na assinatura?
- Um colega afirmou: “vetor é ponteiro”. Dê um exemplo curto que mostra que a afirmação é falsa.

Exercício 14. Strings sem <string.h>.

Implemente à mão, sem usar nenhuma função de <string.h>:

```

1 int meu_tamanho(const char *s);
2 void meu_copia(char *destino, const char *origem);
3 void maiusculas(char *s);
4 int meu_compara(const char *a, const char *b);
```

- Implemente as quatro. `meu_compara` devolve 0 se as strings forem iguais, um valor negativo se a vier antes e positivo se vier depois.
- Para `char nome[20] = "Ana";`, diga o valor de `sizeof nome` e o de `meu_tamanho(nome)`. Por que os dois números são diferentes?
- Onde está o `'\0'` nesse vetor? Desenhe as 20 posições indicando o que se sabe sobre cada uma.
- `meu_copia` tem um risco que `meu_tamanho` não tem. Qual é, e o que o chamador precisa garantir antes de chamá-la?

Exercício 15. Devolvendo mais de um resultado.

Escreva a função

```

1 void estatisticas(const double v[], int n, double *media, double *menor, double *
    maior);
```

- Implemente-a percorrendo o vetor **uma única vez**.
- Escreva a chamada a partir de uma `main` com `double v[] = {3.0, 8.5, 1.25, 6.0};` e imprima os três resultados. Onde aparece o `&` e por quê?

- (c) Por que `v` não precisa de `&` na chamada, mas `media` precisa? Responda usando a regra “C passa tudo por valor”.
- (d) Proteja a função contra `n <= 0` e contra um ponteiro de saída nulo — o chamador que não quiser o maior valor deve poder passar `NULL`.

Exercício 16. Dois erros que compilam sem reclamar.

Análise o programa abaixo.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 char *saudacao(const char *nome) {
5     char buffer[16];
6     strcpy(buffer, "Ola, ");
7     strcat(buffer, nome);
8     return buffer;
9 }
10
11 int main(void) {
12     char *s = saudacao("Maria Aparecida da Silva");
13     printf("%s\n", s);
14     return 0;
15 }
```

- (a) Aponte os **dois** erros distintos e explique o que cada um causa em tempo de execução.
- (b) Por que o compilador aceita esse código (talvez com um aviso) em vez de recusá-lo?
- (c) Reescreva `saudacao` de forma correta, **sem** usar alocação dinâmica. Qual foi a mudança na assinatura?
- (d) Descreva em uma frase como você reescreveria `saudacao` com alocação dinâmica, e diga de quem passa a ser a responsabilidade de liberar a memória.

Parte V — Alocação dinâmica de memória

Exercício 17. Caça às falhas clássicas.

O programa abaixo compila. Ele contém **cinco** defeitos ligados ao uso da memória dinâmica.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 char *duplica(const char *s) {
6     char *novo = malloc(strlen(s));
7     strcpy(novo, s);
8     return novo;
9 }
10
11 int main(void) {
12     char *a = duplica("agenda");
13     printf("%s\n", a);
14     free(a);
15     printf("%s\n", a);
16     free(a);
17
18     int *v = malloc(10 * sizeof(int));
19     v = malloc(20 * sizeof(int));
```

```

20     free(v);
21     return 0;
22 }

```

- Liste os cinco defeitos, um por linha, dizendo em que linha cada um está e a que categoria pertence (vazamento, uso após liberação, liberação dupla, escrita fora do bloco, falta de verificação).
- Reescreva o programa corrigido.
- O que `free(a)` faz com o *bloco* e o que ele faz com o *ponteiro a*? Que hábito de escrita evita o defeito das linhas seguintes?
- Cite as duas ferramentas vistas em aula para detectar esses defeitos e diga, para cada uma, qual dos cinco ela apontaria com mais clareza.

Exercício 18. Um vetor que cresce sozinho.

Implemente uma agenda de inteiros sem limite fixo, com a estrutura e as funções abaixo.

```

1 typedef struct {
2     int *dados;
3     int tamanho;    /* quantos elementos ja foram inseridos */
4     int capacidade; /* quantos cabem no bloco atual */
5 } Agenda;
6
7 int agenda_cria(Agenda *ag, int capacidade_inicial);
8 int agenda_insere(Agenda *ag, int valor);
9 void agenda_destroi(Agenda *ag);

```

- Implemente as três funções. `agenda_insere` deve **dobrar** a capacidade quando o bloco encher, usando `realloc`. As funções que podem falhar devolvem 1 em caso de sucesso e 0 em caso de falha.
- Escreva o *padrão do ponteiro temporário* no `realloc` e explique, em duas frases, o que daria errado se você escrevesse `ag->dados = realloc(ag->dados, ...)` diretamente e a realocação falhasse.
- Depois de um `realloc` bem-sucedido que mudou o bloco de lugar, o que acontece com um ponteiro que o chamador tivesse guardado para `&ag->dados[0]`? Como se chama esse problema?
- Em `agenda_cria`, você usaria `malloc` ou `calloc`? Justifique dizendo o que muda no conteúdo do bloco — e por que “veio zerado no meu teste” não é garantia.
- Escreva uma `main` que crie a agenda com capacidade inicial 2, insira os valores de 1 a 10, imprima quantas vezes a capacidade mudou e destrua a agenda no fim.

Parte VI — Vetores e matrizes dinâmicos

Exercício 19. Matriz como vetor de ponteiros (`double **`).

Uma turma tem `nl` alunos e `nc` avaliações, ambos lidos em tempo de execução.

```

1 double **cria_matriz(int nl, int nc);
2 void libera_matriz(double **m, int nl);

```

- Implemente `cria_matriz` alocando em **dois níveis**: primeiro o vetor de `nl` ponteiros, depois cada linha com `nc` `double`.

- (b) Trate a **falha parcial**: se a alocação da linha k falhar, desfça exatamente o que já havia sido feito e devolva NULL, sem deixar nenhum bloco perdido. Escreva esse trecho por inteiro.
- (c) Implemente `libera_matriz` na ordem correta. Um colega escreveu `free(m)`; antes do laço que libera as linhas. Explique por que isso é um erro e que categoria de defeito ele produz.
- (d) Desenhe a memória para `nl = 3`, `nc = 4`: o vetor de ponteiros e as três linhas. As três linhas ficam necessariamente encostadas na memória? Justifique.

Exercício 20. A mesma matriz em um bloco só.

Agora a mesma turma, com **um único malloc**.

```
1 double *cria_bloco(int nl, int nc);
2 double media_aluno(const double *m, int nl, int nc, int i);
3 double media_avaliacao(const double *m, int nl, int nc, int j);
```

- (a) Implemente `cria_bloco` e escreva a expressão que acessa o elemento da linha i , coluna j .
- (b) Implemente `media_aluno` (média das nc notas do aluno i) e `media_avaliacao` (média das nl notas da avaliação j).
- (c) Na chamada a `malloc`, por que o material insiste em escrever `(size_t) nl * nc * sizeof(double)` em vez de `nl * nc * sizeof(double)`? Descreva a situação em que a diferença aparece.
- (d) Um colega trocou os índices e escreveu `m[j * nc + i]`. O programa continua compilando e, para uma matriz quadrada, continua rodando sem travar. Por que esse é um erro particularmente perigoso?
- (e) Compare as duas representações (`double **` e bloco único) em três aspectos: número de chamadas a `malloc`, contiguidade na memória e facilidade de escrever `m[i][j]`. Diga em que caso você escolheria cada uma.